Dep-Hallucinator: Docker Run & Test Evidence

Updated security modification • 8 October 2026

Two student-level contributions

Modification 1

Persistent allowlist and denylist. Entries added by the user are saved as JSON and survive a new process or container restart.

Modification 2

Secure Maven XML parsing. Malicious external entities in pom.xml files are rejected before they can access local data.

Verified result

63 passed, 37 warnings

The warnings are pre-existing asynchronous mock warnings; every test passed. Docker used Python 3.11-slim.

1. Start Docker and create an isolated container

Run each command separately and wait for it to finish

EXIT 0

```
$ open -a Docker
```

Opens Docker Desktop. Wait until the Docker engine reports that it is running.

EXIT 0

```
$ cd ~/Desktop/dep-hallucinator
docker run -dit --name dep-hallucinator-evidence-20261008 --mount
type=bind,source="$PWD",target=/source,readonly --mount type=bind,source="$HOME/Downloads/Dep-Hallucinator-Test-
Repositories",target=/samples,readonly -w /work python:3.11-slim tail -f /dev/null
```

Creates a temporary Python 3.11 container.
The Desktop repository is read-only at /source.
The sample repositories are read-only at /samples.
/work is the writable test copy.

2. Install the project inside Docker

This installs the new defusedxml security dependency too

EXIT 0

```
$ docker exec dep-hallucinator-evidence-20261008 sh -lc 'cp -a /source/. /work/ && python -m pip install
--disable-pip-version-check -e ".[dev,test]"'

Successfully built dep-hallucinator
Successfully installed ... defusedxml-0.7.1 ... dep-hallucinator-1.0.2
```

What it does: copies the source into the container, installs the project in editable mode, and installs the testing and code-quality tools. Editable mode means tests run the copied source directly.

EXIT 0

```
$ docker exec dep-hallucinator-evidence-20261008 dep-hallucinator --help

Usage: dep-hallucinator [OPTIONS] COMMAND [ARGS]...
Commands: allowlist, batch, cache, config, denylist, forensic, info, scan ...
```

3. Modification 1 — persistent package lists

Simple meaning: a user's list no longer disappears when the command ends

Before this change, add/remove operations changed only Python memory. A second CLI process created a new manager and lost those changes. The modification now writes allowlist.json and denylist.json, loads them at startup, and uses a temporary file plus replace so an interrupted write is less likely to corrupt the real file.

Exact implementation places

src/dep_hallucinator/package_lists.py:5 — required marker
lines 96–150 — JSON read, atomic write, and reload helpers
lines 274–318 and 320–344 — add/remove operations save the result
tests/test_package_lists.py:4–39 — allowlist and wildcard denylist persistence tests

EXIT 0

```
$ docker exec -e PYTHONDONTWRITEBYTECODE=1 dep-hallucinator-evidence-20261008 pytest -p no:cacheprovider
tests/test_package_lists.py

collected 2 items
tests/test_package_lists.py ..                                          [100%]
2 passed in 0.01s
```

4. Modification 1 — manual proof

Add, reload, inspect, remove

EXIT 0

```
$ docker exec dep-hallucinator-evidence-20261008 dep-hallucinator allowlist add docker-safe-package --reason
"Approved Docker demo package"

Added 'docker-safe-package' to allowlist
The entry is written to ~/.config/dep-hallucinator/allowlist.json.
```

EXIT 0

```
$ docker exec dep-hallucinator-evidence-20261008 dep-hallucinator allowlist list

A separate CLI process reloads and displays docker-safe-package.
This proves the data came from disk, not the previous process's memory.
```

EXIT 0

```
$ docker exec dep-hallucinator-evidence-20261008 dep-hallucinator allowlist remove docker-safe-package

Removed 'docker-safe-package' from allowlist
A fresh manager confirms entry_exists_after_remove: False
```

5. Modification 2 — secure Maven XML parsing

Simple meaning: treat every scanned pom.xml as untrusted input

A Maven pom.xml can contain a DOCTYPE and an external entity. With an unsafe XML parser, a crafted dependency name may try to expand data from a local file. The scanner reads dependency files supplied by users, so parsing them defensively improves the real scanning workflow.

What changed

1. pyproject.toml:35 adds defusedxml>=0.7.1.
2. parsers.py:8–10 contains # modification2 and imports the protected parser.
3. parsers.py:780–833 uses that parser and converts blocked XML into a clear ValueError.
4. tests/test_parser_security.py:6–25 builds a malicious pom.xml and proves it is rejected.

Why it is small: the existing Maven parsing logic, dependency extraction, namespaces, and normal error handling stay in place. Only the parser implementation and one focused exception path change.

6. Modification 2 — malicious XML test

The test is safe: it only checks that expansion is blocked

EXIT 0

```
$ docker exec -e PYTHONDONTWRITEBYTECODE=1 dep-hallucinator-evidence-20261008 pytest -p no:cacheprovider
tests/test_parser_security.py

============================= test session starts ==============================
collected 1 item

tests/test_parser_security.py .                                          [100%]

============================== 1 passed in 0.01s ===============================
```

The test writes an external entity named xxe that points to file:///etc/passwd, places &xxe; inside artifactId, and expects “Unsafe XML content”. Passing means the parser stopped the entity instead of expanding it.

Security property tested: the dependency scanner does not expose local file contents through XML external entity expansion.

7. Normal Maven files still work

Security must not break legitimate pom.xml parsing

EXIT 0

```
$ docker exec -e PYTHONDONTWRITEBYTECODE=1 dep-hallucinator-evidence-20261008 pytest -p no:cacheprovider
tests/test_core.py::TestDependencyParsing::test_parse_additional_file_formats
tests/test_core.py::TestDependencyParsing::test_multi_language_file_support

collected 2 items
tests/test_core.py ..                                                    [100%]
2 passed
```

This regression check is important: Modification 2 blocks dangerous XML features, but ordinary Maven dependency elements such as groupId, artifactId, and version are still extracted normally.

Expected behavior

Normal pom.xml → dependency list returned Malicious pom.xml → clear ValueError

8. Run both modifications together

Focused tests first, then the entire project

EXIT 0

```
$ docker exec -e PYTHONDONTWRITEBYTECODE=1 dep-hallucinator-evidence-20261008 pytest -p no:cacheprovider
tests/test_package_lists.py tests/test_parser_security.py

collected 3 items
tests/test_package_lists.py ..                                          [ 66%]
tests/test_parser_security.py .                                         [100%]
3 passed in 0.02s
```

EXIT 0

```
$ docker exec -e PYTHONDONTWRITEBYTECODE=1 dep-hallucinator-evidence-20261008 pytest -p no:cacheprovider

collected 63 items
...............................................................      [100%]
63 passed, 37 warnings in 0.19s
```

Exit code 0 means no test failed.

9. Code-quality checks

Formatting, imports, syntax, and the relevant security scan

EXIT 0

```
$ docker exec dep-hallucinator-evidence-20261008 ruff check --no-cache src/dep_hallucinator/package_lists.py
tests/test_package_lists.py

All checks passed!
```

EXIT 0

```
$ docker exec dep-hallucinator-evidence-20261008 black --check src/dep_hallucinator/package_lists.py
src/dep_hallucinator/parsers.py tests/test_package_lists.py tests/test_parser_security.py

All done!
4 files would be left unchanged.
```

EXIT 0

```
$ docker exec dep-hallucinator-evidence-20261008 bandit -q src/dep_hallucinator/parsers.py

No unsafe XML parser finding. Command completed successfully.
```

10. Real scanner example: original and fake names

The sample contains one legitimate package and one deliberately nonexistent name

EXIT 0

```
$ docker exec dep-hallucinator-evidence-20261008 python -c 'from pathlib import Path; Path("/work/sample-
requirements.txt").write_text("requests==2.32.3\nstudent-demo-package-that-does-not-exist-20261008==1.0.0\n")'

Creates a disposable requirements file inside Docker.
requests is the legitimate/original package.
student-demo-package-that-does-not-exist-20261008 is deliberately fake.
```

EXIT 1

```
$ docker exec dep-hallucinator-evidence-20261008 dep-hallucinator scan sample-requirements.txt --rate-limit 2
--max-concurrent 2

requests                                                LOW / legitimate
student-demo-package-that-does-not-exist-20261008        CRITICAL

Exit code 1 is expected because the sample intentionally contains a critical finding.
```

This sample is test data only; it is not added to the real project dependencies.

11. What does ‘safe percentage’ mean?

A simple classroom summary, not a security guarantee

Dep-Hallucinator assigns each dependency a risk level: LOW, MEDIUM, HIGH, CRITICAL, or ERROR. It also shows suspicion percentages for some individual packages. The program does not define one official safety percentage for a whole repository.

Classroom formula

$$\text{safe percentage} = \text{LOW findings} \div \text{total findings} \times 100$$

This guide treats only LOW as safe. MEDIUM, HIGH, CRITICAL, and ERROR are counted as needing review. ERROR usually means the registry could not be checked, so it must not be counted as safe.

A 100% result means every package received LOW in that scan. It does not prove the application has no vulnerabilities, malicious code, or future supply-chain risk.

12. Ready-made sample repositories

Actual results verified on 8 October 2026 with live PyPI access

`01-safe-python-repo`

requests, numpy, pandas

3 LOW / 3 total

100.00%

`02-mixed-risk-python-repo`

requests + two denylist-pattern names

1 LOW + 2 CRITICAL

33.33%

`03-live-registry-repo`

requests + one nonexistent PyPI name

1 LOW + 1 CRITICAL

50.00%

The third result depends on internet access and registry state. The first two are designed for a predictable classroom demonstration.

13. Scan the sample repositories inside Docker

The extra read-only /samples mount was added to the container command

EXIT 0

```
$ docker exec dep-hallucinator-evidence-20261008 dep-hallucinator scan /samples/01-safe-python-repo/requirements.txt

Verified: requests, numpy, pandas → 3 LOW
Classroom safe percentage: 100.00%
```

EXIT 1

```
$ docker exec dep-hallucinator-evidence-20261008 dep-hallucinator scan /samples/02-mixed-risk-python-repo/requirements.txt

Verified: requests → LOW
ai-student-helper and gpt-dependency-tool → CRITICAL
Classroom safe percentage: 33.33%
```

EXIT 1

```
$ docker exec dep-hallucinator-evidence-20261008 dep-hallucinator scan /samples/03-live-registry-repo/requirements.txt --rate-limit 2 --max-concurrent 2

Verified with PyPI: 1 LOW + 1 CRITICAL
Classroom safe percentage: 50.00%
Needs internet access.
```

14. Save JSON and calculate the percentage

This produces a repeatable number from the scanner's real findings

EXIT 0

```
$ docker exec dep-hallucinator-evidence-20261008 dep-hallucinator scan /samples/02-mixed-risk-python-repo/requirements.txt --output-format json -o /tmp/dep-scan-result.json

Writes every package, risk level, reason, and recommendation to /tmp/dep-scan-result.json inside the container.
```

EXIT 0

```
$ docker exec dep-hallucinator-evidence-20261008 python -c 'import json; d=json.load(open("/tmp/dep-scan-result.json")); f=d["findings"]; low=sum(x["risk_level"]=="LOW" for x in f); print(f"Safe percentage: {100*low/len(f):.2f}% ({low}/{len(f)} LOW)")'

Safe percentage: 33.33% (1/3 LOW)
```

Read the detailed findings too; the percentage is only a compact summary.

15. Exact changed files

These are the only five source/test files changed or added

`src/dep_hallucinator/package_lists.py`

Modification 1: persistence implementation; marker at line 5.

`tests/test_package_lists.py`

Modification 1: two reload/add/remove tests, lines 4–39.

`src/dep_hallucinator/parsers.py`

Modification 2: secure imports at lines 8–10; rejection handling at 824–833.

`pyproject.toml`

Modification 2: defusedxml production dependency at line 35.

`tests/test_parser_security.py`

Modification 2: external-entity rejection test at lines 6–25.

Required source markers: # modification1 and # modification2

16. Cleanup and final result

Remove only the disposable evidence container

```
EXIT 0

$ docker stop dep-hallucinator-evidence-20261008
docker rm dep-hallucinator-evidence-20261008

Stops and deletes the temporary container.
The modified repository on the Desktop is not deleted.
```

Final verified results

Modification 1 focused tests: 2 passed
Modification 1 add/reload/remove and restart persistence: passed
Modification 2 malicious XML rejection: 1 passed
Normal Maven parser regression tests: 2 passed
Both modifications together: 3 passed
Complete project: 63 passed, 37 warnings
Formatting, import, syntax, and relevant security checks: passed

The project is ready for demonstration and submission.